Desafio Técnico – Tech Lead Pleno | Teddy Open Finance

🎯 Objetivo

Entregar um MVP **full-stack** de um sistema de clientes com login, CRUD, listagem, detalhes e admin básico, pronto para rodar localmente via Docker, com observabilidade mínima e arquitetura documentada.

📦 Estrutura do Repositório

O projeto deve ser entregue como **monorepo Nx.dev obrigatório**, contendo pelo menos dois apps:

```
/
  /front-end   → React + Vite + TS
    ├ docker-compose.yml
    ├ .env.example
    └ README.md
  /back-end    → NestJS + TypeORM + PostgreSQL
    ├ docker-compose.yml
    ├ .env.example
    └ README.md
README.md (raiz) → visão geral + arquitetura
```

Cada app deve possuir:

- docker-compose.yml isolado
- .env
- README.md próprio

Na raiz, README.md com visão geral, diagrama da arquitetura (imagem) e instruções gerais.

🧩 Escopo Funcional (MVP)

- Autenticação (e-mail/senha) com JWT
- CRUD de Clientes com soft delete
- Dashboard/Admin simples: totais, últimos clientes e gráfico
- Contador de acessos em detalhe de cliente
- Auditoria com timestamps
- Diferencial: CI/CD, observabilidade

🧱 Requisitos Técnicos

Front-End:

- React + Vite + TS
- UI responsiva
- Roteamento
- Formulários com validação
- Estado global (a critério)
- Testes unitários (diferencial: E2E)
- Docker + docker-compose

Back-End:

- NestJS modular
- TypeORM + Postgres
- JWT
- Validação (class-validator/Zod)
- Swagger obrigatório
- Logs estruturados
- Healthcheck (/healthz)
- Docker + docker-compose
- .env

Monorepo Nx:

- Uso obrigatório do Nx para organizar os apps
- Pipelines separados (frontend/backend) para build e testes

Exemplo de arquitetura:

```
/front-end
  /src
    /app            # rotas/layouts
    /features
      /auth
      /clients
      /dashboard
    /shared         # ui, hooks, libs
  vitest.config.ts
  package.json
  docker-compose.yml   # subir front isolado

/back-end
  /src
    app.module.ts
    /config
    /common         # interceptors, filters, pipes
    /auth           # module, controller, service, dtos, guards
    /clients        # module, controller, service, entity, dtos, repo
    /metrics        # módulo opcional: contador de views
  /migrations
  /prisma | /typeorm  # se preferir camadas
  swagger.ts
  main.ts
  docker-compose.yml   # subir back isolado

README.md
```

🔐 Endpoints Mínimos (Back-End)

- POST /auth/login → autenticação
- POST /clients → cria (auth)
- GET /clients → lista (auth)
- GET /clients/:id → detalhe + contador (auth)
- PUT /clients/:id → atualiza (auth)
- DELETE /clients/:id → soft delete (auth)

- GET /healthz → healthcheck
- Swagger em /docs.

🧭 Fluxos (Front-End)

- Login → redireciona Dashboard
- Dashboard → cards + gráfico + últimos clientes
- Clientes → Listar, Criar, Editar, Excluir (soft delete), Detalhes com contador

☁️ Arquitetura (Visão Local)

- Browser → http://localhost:5173 (FE)
- → API NestJS http://localhost:3000
- → PostgreSQL 5432
- → Redis 6379 (opcional)

☁️ Arquitetura (Visão AWS Proposta)

O objetivo é ilustrar como a aplicação poderia ser implantada em ambiente **cloud** (AWS), considerando escalabilidade, segurança e observabilidade.

🧪 Testes & Qualidade

- Testes unitários obrigatórios em FE e BE
- Testes E2E: diferencial
- ESLint + Prettier
- Commits semânticos
- CI/CD com Nx + GitHub Actions (workflows separados FE/BE)

☁️ Observabilidade

- Logs estruturados obrigatórios (JSON)
- Endpoint /healthz obrigatório
- Endpoint /metrics obrigatório (Prometheus exposition format)
- Opcional: traces (OpenTelemetry/X-Ray)
- README deve explicar por que essas práticas são importantes

🔗 Recursos fornecidos

Design no Figma: Clique aqui

⏱️ Prazo sugerido

Até 3 dias úteis (estimativa de dedicação: 10–18h).

📬 Entrega

O código deve estar em um repositório público (GitHub), com:

- Código-fonte completo
- README raiz com visão geral, instruções, diagrama da arquitetura e explicação sobre escalabilidade.
- Link da aplicação (se houver deploy em cloud)